

Group Topic: Predicting Student Dropout Risk Using Academic and Behavioral Data Lab No.

Git Repo/ Colab Link:

Date:04-24-26

<https://github.com/dgbb-prog/student-dropout-bigdata-datacleaning>

Objectives

The objective of this weekly task is to perform data cleaning and preprocessing on the Open University Learning Analytics Dataset (OULAD). This includes identifying and handling missing values, removing duplicate and inconsistent records, detecting outliers, and transforming raw data into a structured and usable format.

The purpose of this task is to prepare high-quality data for subsequent analysis and modeling, and to gain practical experience in preprocessing large-scale educational datasets.

Tasks Completed

1. Load the Dataset into DataFrames

- Imported multiple CSV files and loaded them into DataFrames for processing.

```
student_info = pd.read_csv('studentInfo.csv')
student_vle = pd.read_csv('studentVle.csv')
student_assessment = pd.read_csv('studentAssessment.csv')
```

This step initializes the datasets and prepares them for analysis.

2. Perform Exploratory Data Analysis (EDA)

- Checked dataset structure, column names, and row counts
- Identified missing values across datasets

```
student_info.isnull().sum()
```

This step helps identify data quality issues such as missing or incomplete values.

3. Clean Demographic Data (studentInfo)

- Removed duplicate records
- Filled missing categorical values (imd_band) with "Unknown"
- Dropped rows with missing critical columns

```
student_info_clean = student_info.drop_duplicates()
student_info_clean['imd_band'] = student_info_clean['imd_band'].fillna('Unknown')
student_info_clean = student_info_clean.dropna(subset=['studied_credits'])
```

This ensures that demographic data is consistent and usable.

4. Feature Engineering

- Created new variables to support classification:

Binary Dropout Label

```
student_info_clean['is_dropout'] = (student_info_clean['final_result'] == 'Withdrawn').astype(int)
```

Risk Level Classification

```
def classify_risk(result):
    if result == 'Withdrawn':
        return 'High Risk'
    elif result == 'Fail':
        return 'Medium Risk'
    else:
        return 'Low Risk'
```

- Applied classification to categorize students into risk levels:
 - High Risk (Withdrawn)
 - Medium Risk (Fail)
 - Low Risk (Pass)

5. Clean Academic Data (studentAssessment)

- Removed duplicate records
- Dropped rows with missing score
- Filtered scores within valid range (0–100)

```
assessment_clean = student_assessment.drop_duplicates()
assessment_clean = assessment_clean.dropna(subset=['score'])
assessment_clean = assessment_clean[
    (assessment_clean['score'] >= 0) & (assessment_clean['score'] <= 100)
]
```

This step ensures accuracy and validity of academic performance data.

6. Clean Behavioral Data (studentVle)

- Removed duplicate records
- Removed rows with null or zero sum_click values
- Applied Interquartile Range (IQR) method to remove outliers

```
Q1 = vle_clean['sum_click'].quantile(0.25)
Q3 = vle_clean['sum_click'].quantile(0.75)
```

```
IQR = Q3 - Q1  
vle_clean = vle_clean[vle_clean['sum_click'] <= Q3 + 3 * IQR]
```

This step improves the reliability of behavioral indicators.

7. Aggregate Behavioral and Academic Data

- Summarized student activity and performance:

```
vle_agg = vle_clean.groupby(['id_student']).agg(  
    total_clicks=('sum_click', 'sum')  
) .reset_index()
```

This provides meaningful features such as total activity per student.

8. Merge Datasets

- Combined datasets using common key (id_student) to create a master dataset.

```
master_df = student_info_clean.merge(vle_agg, on='id_student', how='left')
```

This integrates demographic, academic, and behavioral data into one dataset.

9. Validate and Save Final Dataset

- Checked dataset structure and ensured no missing values
- Saved cleaned dataset for further analysis

```
master_df.to_csv('oulad_cleaned_master.csv', index=False)
```

Tasks to be Completed

Results and Analysis

Loaded them into DataFrames for processing.

```
[ ]      print('studentInfo      : {student_info.shape[0]:,} rows x {student_info.shape[1]} columns')
print(f'studentVle      : {student_vle.shape[0]:,} rows x {student_vle.shape[1]} columns')
print(f'studentAssessment : {student_assessment.shape[0]:,} rows x {student_assessment.shape[1]} columns')
print(f'studentReg      : {student_reg.shape[0]:,} rows x {student_reg.shape[1]} columns')
print(f'assessments      : {assessments.shape[0]:,} rows x {assessments.shape[1]} columns')
print(f'vle              : {vle.shape[0]:,} rows x {vle.shape[1]} columns')
print(f'courses          : {courses.shape[0]:,} rows x {courses.shape[1]} columns')

total_rows = (student_info.shape[0] + student_vle.shape[0] +
              student_assessment.shape[0] + student_reg.shape[0] +
              assessments.shape[0] + vle.shape[0] + courses.shape[0])
print(f'\nTOTAL ROWS (all files combined): {total_rows:,}')

▼      === DATASET LOADED SUCCESSFULLY ===
studentInfo      : 32,593 rows x 12 columns
studentVle       : 10,655,280 rows x 6 columns
studentAssessment : 173,912 rows x 5 columns
studentReg       : 32,593 rows x 5 columns
assessments      : 206 rows x 6 columns
vle              : 6,364 rows x 6 columns
courses          : 22 rows x 3 columns

TOTAL ROWS (all files combined): 10,900,970
```

Identified missing values in each dataset

```
[ ]      missing = missing[missing > 0]
if len(missing) > 0:
    print(f'\n{name}:')
    print(missing)
else:
    print(f'\n{name}: No missing values found.')

▼      === MISSING VALUES PER FILE ===

studentInfo:
imd_band    1111
dtype: int64

studentVle: No missing values found.

studentAssessment:
score      173
dtype: int64

studentRegistration:
date_registration      45
date_unregistration    22521
dtype: int64
```

Apply Data Cleaning on Academic Data (studentAssessment)

```
[ ]      # Step 4: Fill missing date_submitted with median
median_date = assessment_clean['date_submitted'].median()
assessment_clean['date_submitted'] = assessment_clean['date_submitted'].fillna(median_date)

print(f'studentAssessment rows AFTER cleaning: {assessment_clean.shape[0]:,}')
print()
print('Score statistics:')
print(assessment_clean['score'].describe())

▼      studentAssessment rows BEFORE cleaning: 173,912
studentAssessment rows AFTER cleaning: 173,739

Score statistics:
count      173739.000000
mean         75.799573
std          18.798107
min           0.000000
25%          65.000000
50%          80.000000
75%          90.000000
max         100.000000
Name: score, dtype: float64
```

Apply Data Cleaning on Demographic Data (studentInfo)

```
[ ]      return 'Low Risk'

      student_info_clean['risk_level'] = student_info_clean['final_result'].apply(classify_risk)

      print(f'\nstudentInfo rows AFTER cleaning: {student_info_clean.shape[0]:,}')
      print(f'New columns added: is_dropout, risk_level')
      print()
      print('Risk Level Distribution:')
      print(student_info_clean['risk_level'].value_counts())

▼      studentInfo rows BEFORE cleaning: 32,593
      After removing duplicates: 32,593
      After dropping critical nulls: 32,593

      studentInfo rows AFTER cleaning: 32,593
      New columns added: is_dropout, risk_level

      Risk Level Distribution:
      risk_level
      Low Risk      15385
      High Risk     10156
      Medium Risk   7052
      Name: count, dtype: int64
```

Apply Data Cleaning on Behavioral Data (studentVle)

```
[ ]      IQR = Q3 - Q1
      upper_bound = Q3 + 3 * IQR
      vle_clean = vle_clean[vle_clean['sum_click'] <= upper_bound]

      print(f'studentVle rows AFTER cleaning: {vle_clean.shape[0]:,}')
      print()
      print('Sum click statistics after cleaning:')
      print(vle_clean['sum_click'].describe())

▼      studentVle rows BEFORE cleaning: 10,655,280
      studentVle rows AFTER cleaning: 9,398,040

      Sum click statistics after cleaning:
      count      9.398040e+06
      mean        2.625069e+00
      std          2.473275e+00
      min          1.000000e+00
      25%          1.000000e+00
      50%          2.000000e+00
      75%          3.000000e+00
      max          1.300000e+01
      Name: sum_click, dtype: float64
```

Save Final Cleaned Dataset

```
▼      =====
      DATA CLEANING SUMMARY REPORT
      =====
      Dataset : OULAD (Open University Learning Analytics)
      Source  : https://analyse.kmi.open.ac.uk/open-dataset
      License : CC-BY 4.0 (Open, Ethical)

      Files Loaded:
      studentInfo.csv      : 32,593 rows
      studentVle.csv       : 10,655,280 rows
      studentAssessment.csv : 173,912 rows
      studentRegistration.csv : 32,593 rows
      assessments.csv      : 206 rows
      vle.csv              : 6,364 rows
      courses.csv          : 22 rows

      Total rows (all files) : 10,900,970
      Minimum requirement    : 500,000 rows
      Requirement met        : YES ✓

      Cleaning Steps Applied:
      1. Removed duplicate rows
      2. Handled missing values (fillna / dropna)
      3. Removed invalid score values (outside 0-100)
```

```
assessments.csv      : 206 rows
vle.csv              : 6,364 rows
courses.csv          : 22 rows

Total rows (all files) : 10,900,970
Minimum requirement    : 500,000 rows
Requirement met         : YES ✓

Cleaning Steps Applied:
1. Removed duplicate rows
2. Handled missing values (fillna / dropna)
3. Removed invalid score values (outside 0-100)
4. Removed zero/null VLE clicks
5. Removed outliers using IQR method
6. Created is_dropout binary label
7. Created risk_level classification column
8. Merged all files into one master dataset

Final Master Dataset   : 32,593 rows x 19 cols
Saved as               : oulad_cleaned_master.csv

Visualizations Generated : 8 charts saved as PNG
=====
```

Key findings:

- Missing values were effectively handled
- Invalid and inconsistent records were removed
- Behavioral data revealed significant variation in student engagement
- Outlier removal improved data reliability

The cleaned dataset is now suitable for:

- Student risk classification (High, Medium, Low)
- Dropout prediction modeling
- Behavioral pattern analysis

Challenges and Solutions

Challenge 1: Multiple datasets with different formats

Solution: Identified common keys and merged datasets properly

Challenge 2: Missing and inconsistent values

Solution: Applied data cleaning techniques such as `dropna()` and `fillna()`

Challenge 3: Presence of outliers

Solution: Used IQR method to remove extreme values

Conclusion

The objectives of the weekly task were successfully achieved. The dataset was cleaned, processed, and transformed into a structured format suitable for analysis and modeling.

This task provided valuable insights into:

- The importance of data preprocessing
- Handling large-scale datasets
- Feature engineering techniques

- Ensuring data quality for predictive analytics

The processed dataset will support further tasks such as risk classification, behavioral analysis, and dropout prediction.